

Массивы

1. Выделить место под одномерный массив, все элементы которого нули.
2. Выделить место под двумерный массив размером $n \times m$.
3. Объявить одномерный массив и посчитать в нём количество нулей (единиц)
4. Длина массива
5. Написать массив и найти в нем макс/мин элемент.
6. Сумма элементов массива
7. Описать двумерный массив и переслать в регистр AX элемент массива, стоящий на [3][2]
8. Сумму каждой строки в двумерном массиве

Строки

1. Вывод строки
2. Найти в строке первую звездочку
3. Найти в строке последнюю звездочку
4. Занести звездочку в строку с помощью MOVS
5. Записать в строку s1 строку s2 с помощью команды MOVS
6. Часть строки s1 занести в s2
7. Копирование s2 в s1

Работа с файлами

1. Создать и открыть файл
2. В нем переместить указатель на конец
3. Или переместить указатель на начало
4. Или же от текущей позиции указателя сместиться на сколько-то (здесь 1)
5. Открыть и закрыть файл
6. Переименовать и закрыть файл
7. Открыть и переименовать
8. Считать символ с клавиатуры и записать его в строку
9. Организуйте ввод строки из консоли, используя функции работы с файлами.

Макросы

1. Пример использования макроса
2. Вывести символ на экран
3. Вывести строку на экран
4. Посчитать сумму двух чисел
5. Макрос, определяющий номер функции прерывания
6. Макрос для помещения в стек регистров из списка - блок внутри макроса

Другое

1. Описать структуру
2. Написать процедуру и обращение к ней и возврат
3. Реализация цикла loop - попроще
4. Вывести сумму элементов
5. Написать программу с использованием логических операций (попроще)
6. Максимум из двух чисел
7. Написать программу с использованием арифметических операций (попроще)
8. Переслать в косвенно-регистровый операнд непосредственный
9. Написать команды, реализующие вычисления по формуле $20/3$
10. Написать программу вывода с помощью точечной директивы

Массивы

1. Выделить место под одномерный массив, все элементы которого нули.

array db 100 dup (1) ; 1-мерный массив, размером 100, заполненный единицами

2. Выделить место под двумерный массив размером $n*m$. array db 5 DUP (6 Dup (?)); Двумерный массив размером $5*6$

3. Объявить одномерный массив и посчитать в нём количество нулей (единиц)

array db 100 dup (1) ; 1-мерный массив, размером 100 mov cx, 100 n:

cmp array[cx], 1

jne s

inc ax

s:

loop n

4. Длина массива

array dw 1, 2, 4, 5, 9

len db (\$-array) / type array

5. Написать массив и найти в нем макс/мин элемент.

array db 1,2,3,4,5; Байтовый массив

mov dx, 1; Индекс от куда начинаем сравнивать

mov ax, array[0]; Задаем нулевой элемент как максимальное число

s:

cmp dx, 5; Если счетчик вышел за пределы массива

jz exit; То выходим из программы, результат хранится в ax

cmp ax, array[dx]; Если текущий максимальный элемент больше или равен элементу

;из массива под индексом dx

jge n; То мы

mov ax, array[dx]; Если меньше, то записываем в ax новое максимальное значение

n:

inc dx; Увеличиваем индекс

jmp s; И переходим в начало

exit:

6. Сумма элементов массива

.model small

.stack 100h

.386 .data

array dw 1,2,3,4,5

.code start:

mov ax, @data

mov ds, ax

mov cx, 5

xor ax, ax

mov si, 0

arrSum:

add ax, array[si]

add si, 2

loop arrSum

7. Описать двумерный массив и переслать в регистр AX элемент массива, стоящий на [3][2]

mas db 20 dup (10 dup (?))

mov bx, 3

mov di, 2

mov ax, mas[bx][di]

8. Сумму каждой строки в двумерном массиве

```
mov cx, 5 ; количество
str:
  push cx
  xor si, si
  mov cx, 6 ; количество столбцов
elem:
  add ax, mas[bx][si]
  inc si
  loop elem
pop cx
inc bx
push ax
xor ax, ax
loop str
```

Строки

1. Вывод строки

```
text db 'I am KING!$'

mov dx, text; Записываем адрес строки
mov ah, 9h Вывод строки
int 21h; Прерывание

mov ax, 4C00h; Выход из программы
int 21h; Прерывание
```

2. Найти в строке первую звездочку

```
cld ; DF=0
push ds
pop es
lea di, string
mov cx, 20
mov al, '*'
repe scasb ; пока не нашли совпадение
jne finish ; Если таки не нашли – конечная...
finish:
```

3. Найти в строке последнюю звездочку

```
std ; DF=1
push ds
pop es
lea di, string
mov cx, 20
mov al, '*'
repe scasb ; while (al != str[i]); повторяем пока ZF !=1
jne finish ; Если таки не нашли – конечная...
finish:
```

4. Занести звездочку в строку с помощью MOVS

```
S1 db ?
S2 db "*"
cld ; DF=0
lea SI, S2
push ds
pop es
lea di, S1
mov cx, 3
rep movs
```

5. Записать в строку s1 строку s2 с помощью команды MOVS

```
S1 db "hello$"  
S2 db "world$"  
cld  
lea SI, S2  
push ds  
pop es  
lea DI, S1  
mov CX, 5 ;длина строки S2  
rep movs
```

6. Часть строки s1 занести в s2

```
S1 db "hello world$"  
S2 db ?  
cld  
lea SI, S1  
push ds  
pop es  
lea DI, S2  
mov CX, 5 ;длина строки S1  
met:  
    lodsb ; DS:SI → AL  
    stosb ; AL → ES:DI
```

7. Копирование s2 в s1

```
S1 db "hello$"  
S2 db "world$"  
cld  
lea SI, S2 ;вторую строку запишем в источник  
push ds  
pop es  
lea DI, S1 ;первую строку запишем в приемник  
mov CX, 10 ;сумма двух строчек  
rep movs
```

Работа с файлами

1. Создать и открыть файл

```
mov ah, 3Ch  
mov dx, offset specfail  
mov cx, 0  
int 21h  
mov ah, 3dh  
mov al, 02h  
mov dx, 80h+1Eh  
int 21h  
mov ah, 5Bh  
mov cx, 0  
mov dx, specfail  
int 21h
```

Открытие
файла

2. В нем переместить указатель на конец

```
mov ah, 42h  
mov al, 02h  
mov cx, 0  
mov dx, cx  
int 21h
```

3. Или переместить указатель на начало

```
mov ah, 42h  
mov al, 00h  
mov cx, 0  
mov dx, cx  
int 21h
```

4. Или же от текущей позиции указателя сместиться на сколько-то (здесь 1)

```
mov ah, 42h ;переместить указатель  
mov al, 01h ;от текущей позиции  
dec cx ;назад  
dec cx ;на 1  
mov dx, cx  
int 21h
```

5. Открыть и закрыть файл

```
mov ah, 3dh  
mov al, 02h  
mov dx, 80h+1Eh  
int 21h  
mov ah, 3Eh  
int 21h
```

6. Переименовать и закрыть файл

```
mov ah, 56h  
mov dx, old  
mov di, new  
int 21h  
mov ah, 3Eh  
int 21h
```

7. Открыть и переименовать

```
mov ah, 3dh  
mov al, 02h  
mov dx, 80h+1Eh  
int 21h  
mov ah, 56h  
mov dx, old  
mov di, new  
int 21h
```

8. Считать символ с клавиатуры и записать его в строку

```
mov ah, 3Fh  
mov bx, 0  
mov cx, 1  
mov dx, offset string  
int 21h
```

9. Организуйте ввод строки из консоли, используя функции работы с файлами.

```
mov ah, 3fh  
mov bx, 0  
mov cx, 2  
int 21h
```

Макросы

1. Пример использования макроса

```
Sample MACRO a, b
    mov dx, a
    mov ax, b
endm
add ax, 5
Sample cx, ax
inc dx
```

2. Вывести символ на экран

```
M macro char
mov dl, offset char
mov ah, 02h
endm
```

3. Вывести строку на экран

```
M macro string
mov dx, offset string
mov ah, 09h
endm
```

4. Посчитать сумму двух чисел

```
sum macro n1, n2
add n1, n2
endm
```

5. Макрос, определяющий номер функции прерывания

```
dos MACRO func
mov ah, func
int 21h
ENDM
```

```
; Тогда предыдущий
message MACRO string
mov dx, OFFSET string
dos 09h
ENDM
```

6. Макрос для помещения в стек регистров из списка - блок внутри макроса

```
push_regs MACRO reg_list
    IRP reg, <reg_list>
        push reg
    ENDM
ENDM
```

```
; Тогда обращение к такой макрокоманде
push_regs <ax, cx, si, di>
```

1. Описать структуру

```
TESTSTRUCT struct; Объявляем структуру
one db ?
two dw 1
TESTSTRUCT ends

tz TESTSTRUCT<25, 10753>; Объявляем переменную с типом
;структуры и заносим значения 25 в one и 10753 в two

mov ax, [tz.one]; Записываем в ax значение из tz.one,
;то есть в ax, будет записано 25
```

2. Написать процедуру и обращение к ней и возврат

```
function proc
    mov ax, 1;
    add ax, dx;
    ret;
function endp

mov dx, 5
call function
```

3. реализация цикла loop - попроще

```
mov dx, 0
mov cx, 10; Задаем значения счетчика от которого начнем отсчет
s:
inc dx; Увеличиваем dx на 1
loop s; Выполняем пока cx не будет равен 0
```

4. Вывести сумму элементов

```
array db 1,2,3,4,5; Байтовый массив

mov bx, 0; В bx будет храниться сумма всех чисел массива
mov cx, 5; Счетчик начинаем с последнего элемента
s:
add bx, array[cx]; Складываем число из массива с числом в регистре bx
loop s; продолжаем пока счетчик не будет равен 0

mov ax, bx; записываем число
mov cx, 10; система счисления
div cl; делим
add ax, '0' ; прибавляем аски символ 0

mov dx, ax; указатель на строку
mov ah, 9; Вывод строки
int 21h; Прерывание
```

5. Написать программу с использованием логических операций (попроще)

```
mov ax, 20
mov bx, 3

add ax, 7 ; прибавляем ax + 7
div bx ; делим ax на bx
mul ax ; умножаем ax на ax
sub ax, bx ; вычитаем ax - bx
```

6. Максимум из двух чисел

```
; вызывать это вот так
mov ax, 8
mov bx, 15
call max
; сама процедура
max proc
    cmp ax, bx
    jg exit ;если ax больше bx
    mov ax, bx
exit:
    ret
max endp
```

7. Написать программу с использованием арифметических операций (попроще)

```
mov ax, 20
mov bx, 3
add ax, 7 ; прибавляем ax+7
div bx ; делим ax на bx
mul ax ; умножаем ax на ax
sub ax, bx ; вычитаем ax-bx
```

8. Переслать в косвенно-регистровый операнд непосредственный

```
mov byte ptr [si], 12 ;((DS = ptr)+(SI = index)) = 12
```

9. Написать команды, реализующие вычисления по формуле $20/3$

```
mov ax, 20
mov bl, 3
div bl
; ah целая часть, al остаток
```

10. Написать программу вывода с помощью точечной директивы

```
.MODEL
.STACK
.DATA
```

```
St1 DB 'Line1', '$'
St2 DB 'Line2', '$'
St3 DB 'Line3', '$'
```

```
.CODE
```

```
begin: MOV AH, 9; номер функции вывода строки на экран
MOV DX, offset St1; адрес, который содержится в регистре DX Int 21h
MOV DX, offset St2
Int 21h
MOV AH, 4CH
Int 21h
```

```
end begin
```

'\$'определяет конец строки, которую необходимо вывести на экран в результате выполнения программы на экране будет: Line1
Line2 Line3.

Если необходимо вывести результат в три строки

```
Line1
Line2 Line3,
то в сегмент данных необходимо внести изменения
St1 DB 'Line1', 13, 10, '$'
St2 DB 'Line2', 0Dh, 0Ah, '$'
St3 DB 'Line3', '$'
```

Здесь константы 13 и 10, или в шестнадцатеричном формате 0Dh и 0Ah осуществляют перевод строки и возврат каретки, т.е. переход в начало следующей строки.